

Relatório Técnico

1. Visão geral

API Back-End para a rede Raízes do Nordeste.

2. Organização do relatório

1. Introdução
2. Objetivos
3. Análise do problema
4. Requisitos funcionais
5. Requisitos não funcionais
6. Priorização do MVP
7. Casos de uso
8. DER
9. Diagrama de classes
10. Diagrama de sequência
11. Arquitetura da solução
12. API e endpoints
13. Segurança e LGPD
14. Pagamento mock
15. Plano de testes
16. Artefatos do projeto
17. Conclusão
18. Referências

3. Introdução

Este relatório descreve uma API REST para uma rede de lanchonetes em expansão, adaptada ao estudo de caso de atendimento em ambiente de restaurante. A solução centraliza pedidos vindos de diferentes canais e organiza funcionalidades essenciais para uma operação realista: autenticação, perfis de acesso, cardápio por unidade, estoque, pagamento simulado, fidelidade e auditoria.

A proposta resolve um problema comum em restaurantes e lanchonetes: a fragmentação entre pedidos presenciais, totens, aplicativos e retirada agendada. Sem uma API central, cada canal tende a manter dados próprios, dificultando o controle de estoque, o acompanhamento do preparo e a análise de demanda.

4. Objetivos

O objetivo principal é implementar uma API funcional com autenticação, perfis, produtos, unidades, estoque, pedidos, pagamento mock, fidelidade e auditoria.

Objetivos específicos:

- Modelar entidades principais do domínio de restaurante.
- Implementar endpoints REST com validação de entrada e respostas padronizadas.
- Proteger rotas com JWT e autorização por perfil.
- Registrar a origem de cada pedido por meio de canalPedido.
- Simular pagamento aprovado e recusado sem depender de provedor externo.
- Demonstrar impacto de pagamento recusado ou pedido cancelado no estoque reservado.
- Documentar a API com Swagger, Postman, Mermaid e textos técnicos.
- Validar regras centrais com testes unitários e e2e.

5. Análise do problema

A empresa precisa registrar pedidos de APP, TOTEM, BALCÃO, PICKUP e WEB. Sem um back-end centralizado, o controle de estoque, status e origem dos pedidos fica disperso.

No contexto de atendimento, a dispersão causa filas maiores, baixa previsibilidade de preparo, risco de venda de itens indisponíveis e dificuldade para medir quais canais são mais usados. A API proposta atua como camada central de integração: todos os canais enviam pedidos para a mesma base, e a cozinha/operação acompanha status, pagamento e estoque em um fluxo único.

6. Requisitos funcionais

- Login e cadastro.
- Controle de usuários por perfil.
- Gestão de unidades, produtos, cardápio e estoque.
- Pedido com canalPedido obrigatório.
- Pagamento mock aprovado ou recusado.
- Fidelidade por consentimento.
- Auditoria de ações sensíveis.

7. Requisitos não funcionais

- API REST.
- Banco PostgreSQL.
- ORM Prisma.
- JWT e bcrypt.
- Swagger.
- Erro padronizado.
- Testes unitários e e2e com Jest.

- Execução local com Docker Compose.
- Documentação técnica.

8. Priorização do MVP

O MVP prioriza o fluxo Pedido -> Pagamento mock -> Atualização de status, pois ele demonstra a regra central da operação e conecta autenticação, estoque, pagamento, fidelidade e auditoria.

9. Casos de uso

O diagrama está em docs/diagramas/casos-de-uso.mmd.

10. DER

O diagrama entidade-relacionamento está em docs/diagramas/der.mmd.

11. Diagrama de classes

O diagrama de classes está em docs/diagramas/classes.mmd.

12. Diagrama de sequência

O fluxo de pedido e pagamento está em docs/diagramas/sequencia-pedido-pagamento.mmd.

13. Arquitetura da solução

A API usa NestJS com módulos por domínio. O Prisma isola o acesso ao banco, enquanto guards e decorators controlam autenticação e autorização.

A organização em camadas separa responsabilidades:

- api: controllers e DTOs responsáveis pelo contrato HTTP.
- application: services com regras de negócio e orquestração transacional.
- domain: regras puras reaproveitáveis, como cálculos de fidelidade.
- common: filtros, guards e decorators compartilhados.
- prisma: conexão e persistência.

Essa divisão facilita manutenção, testes e evolução para novos canais ou provedores externos.

14. API e endpoints

A lista de endpoints está documentada em docs/endpoints.md e no Swagger em /api/docs.

15. Segurança e LGPD

A solução aplica hash de senha, token JWT, controle de perfis, minimização de dados e consentimento para fidelidade. Mais detalhes estão em docs/lgpd-seguranca.md.

16. Pagamento mock

O pagamento mock simula os resultados APROVADO e RECUSADO. Em caso de recusa, o pedido muda para PAGAMENTO_RECUSADO e o estoque reservado é restaurado. Pagamentos só podem ser processados enquanto o pedido está aguardando pagamento.

17. Plano de testes

O plano está em docs/plano-testes.md e a collection Postman contém cenários positivos e negativos. A validação automatizada inclui:

- Testes unitários para regras de fidelidade.
- Testes e2e para login, cadastro público, autorização, criação de pedido, pagamento, estoque, cancelamento e filtro por canal.
- Build TypeScript para detectar erros de tipagem.
- Validação do schema Prisma.

18. Artefatos do projeto

O repositório inclui código NestJS, Prisma schema, migration, seed, Docker Compose, Swagger, Postman, diagramas Mermaid, testes automatizados e documentação. O pacote limpo pode ser gerado com:

```
npm run package:delivery
```

O arquivo compactado remove dependências instaladas, build local, arquivos temporários e arquivos locais de validação, mantendo apenas os artefatos necessários para execução, análise técnica e testes.

19. Conclusão

O projeto atende ao fluxo principal proposto e oferece uma base clara para evoluir para uma aplicação de mercado. A API demonstra como centralizar pedidos multicanal, proteger operações por perfil, reservar estoque no momento do pedido, processar pagamento simulado e registrar auditoria das ações sensíveis.

Como evolução, a solução pode receber pagamento real, cupons de fidelidade, painel administrativo, relatórios por canal, notificações de preparo e integração com dispositivos de autoatendimento.

20. Referências

- Documentação oficial do NestJS.
- Documentação oficial do Prisma.

- Documentação oficial do PostgreSQL.
- Documentação oficial do Swagger/OpenAPI.

Verificação Técnica

Verificação Técnica

Use esta lista para validar execução local, testes, documentação e empacotamento do projeto.

Ambiente

- ☐ Copiar .env.example para .env, quando rodar fora do Docker Compose.
- ☐ Executar npm install.
- ☐ Executar npm run prisma:generate.
- ☐ Subir banco e API com docker compose up --build -d.
- ☐ Rodar seed com docker compose exec api npm run prisma:seed.
- ☐ Abrir Swagger em <http://localhost:3000/api/docs>.

Validação técnica

- ☐ Executar npm run lint.
- ☐ Executar npm run build.
- ☐ Executar npm test.
- ☐ Executar docker compose exec api npm run test:e2e.
- ☐ Conferir login válido em POST /auth/login.
- ☐ Conferir que cadastro público não aceita role.
- ☐ Conferir criação de pedido com canalPedido.
- ☐ Conferir pagamento aprovado.
- ☐ Conferir pagamento recusado com estoque restaurado.
- ☐ Conferir cancelamento com estoque restaurado.
- ☐ Conferir bloqueio de pagamento de pedido de outro cliente.

Documentação

- ☐ Ler README.md.
- ☐ Conferir docs/endpoints.md.
- ☐ Conferir docs/api.md.
- ☐ Conferir docs/auth.md.
- ☐ Conferir docs/seguranca.md.
- ☐ Conferir docs/plano-testes.md.
- ☐ Conferir docs/relatorio-tecnico.md.
- ☐ Conferir Swagger/OpenAPI em /api/docs.

Postman

- ☐ Importar postman/raizes-do-nordeste.postman_environment.json.
- ☐ Importar postman/raizes-do-nordeste.postman_collection.json.
- ☐ Rodar T01 para salvar accessToken.
- ☐ Rodar T04 para salvar pedidold.
- ☐ Rodar T09 para pagamento aprovado.
- ☐ Rodar T10A e T10B para pagamento recusado em pedido separado.

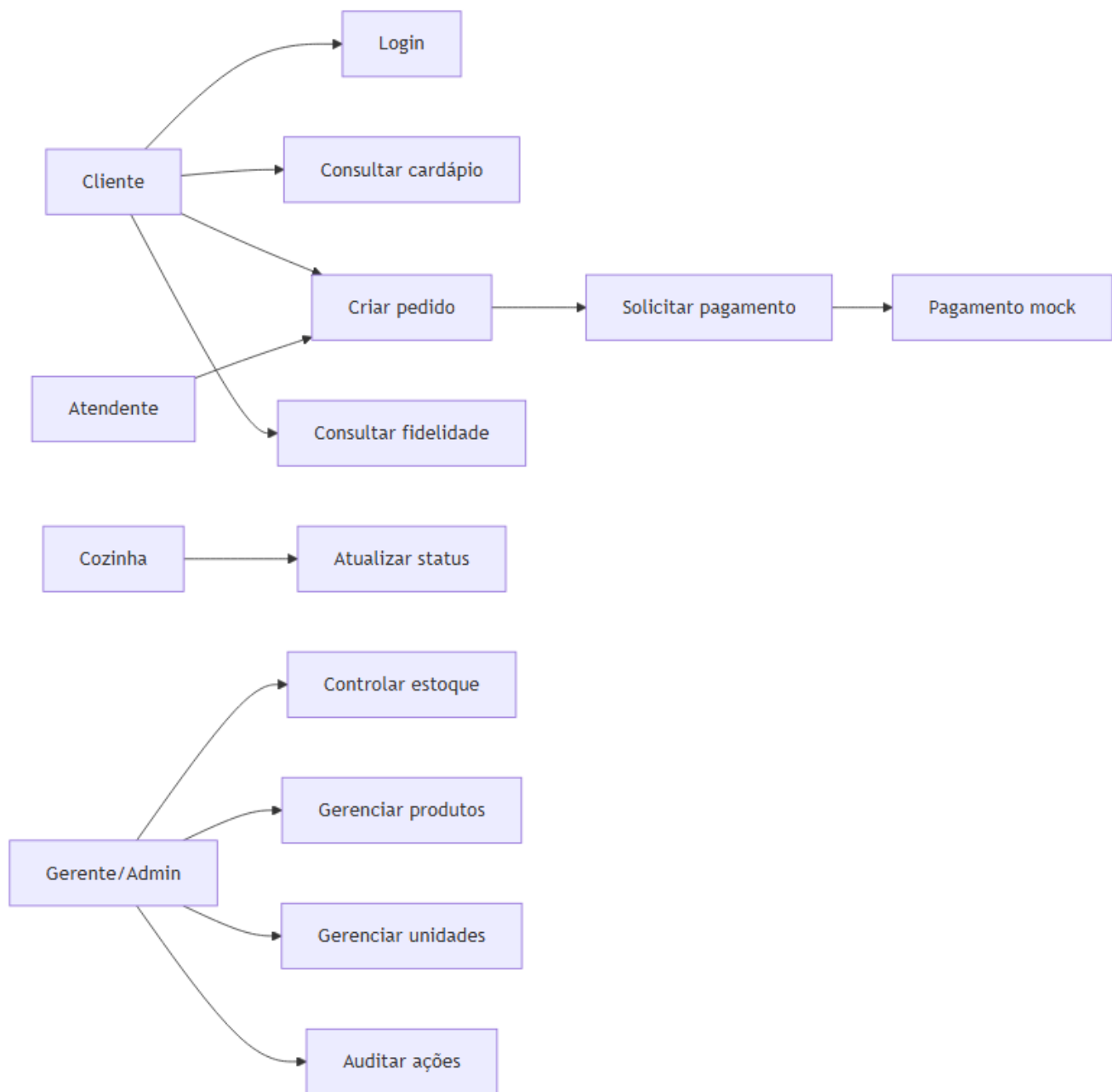
Diagramas e relatório

- ☐ Conferir imagens em docs/diagramas/renderizados.
- ☐ Conferir DER em docs/diagramas/renderizados/der.png.
- ☐ Conferir DER em docs/diagramas/renderizados/der.pdf.
- ☐ Conferir PDF técnico em docs/pdf/relatorio-tecnico.pdf.

Pacote

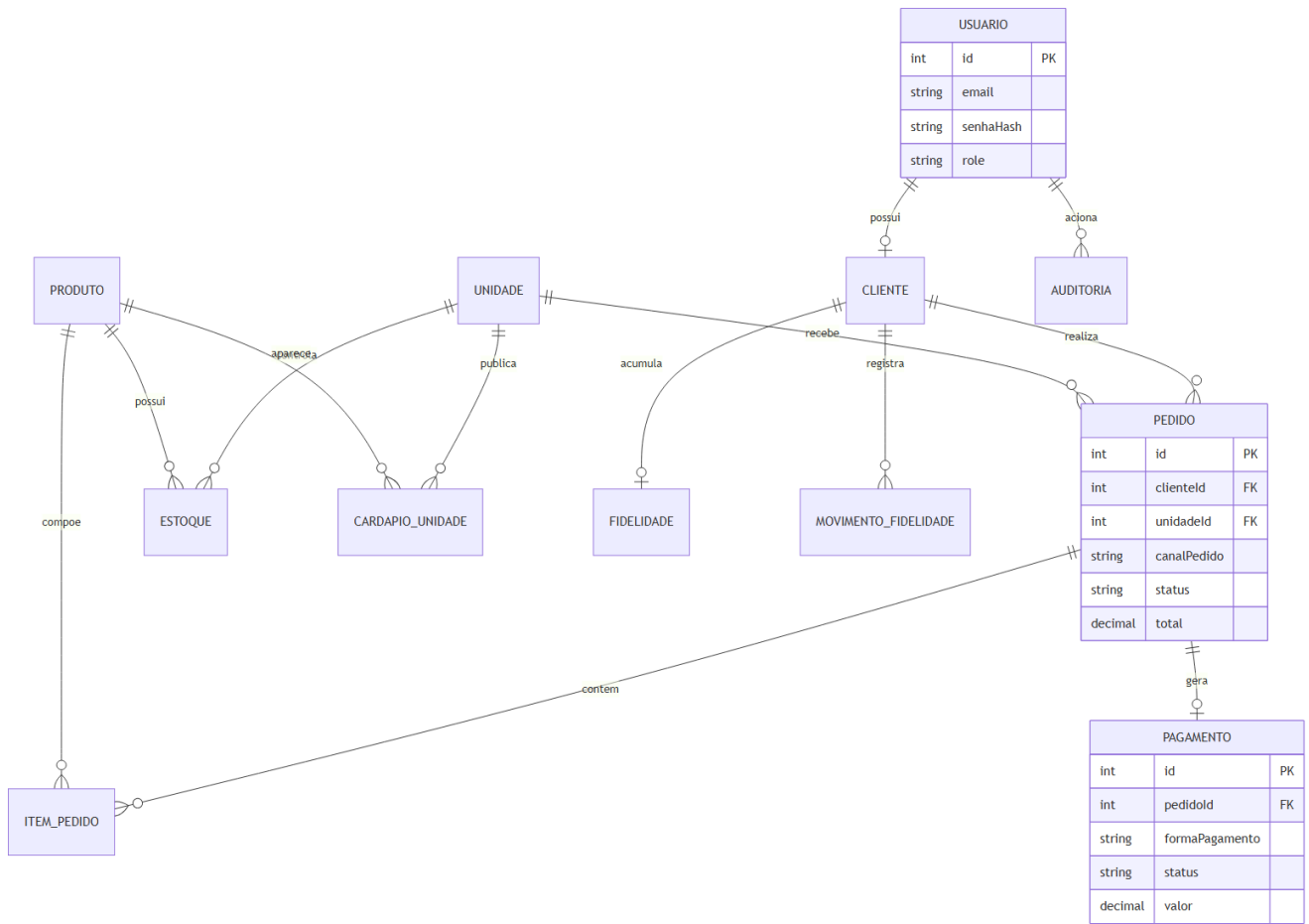
- ☐ Executar npm run package:delivery.
- ☐ Conferir pacote/raizes-do-nordeste-api.zip.
- ☐ Verificar que o pacote não inclui node_modules, dist, .env, temporários ou arquivos locais de validação.

Diagrama de Casos de Uso



Arquivo: docs/diagramas/renderizados/casos-de-uso.png

Diagrama Entidade-Relacionamento



Arquivo: docs/diagramas/renderizados/der.png

Diagrama de Classes

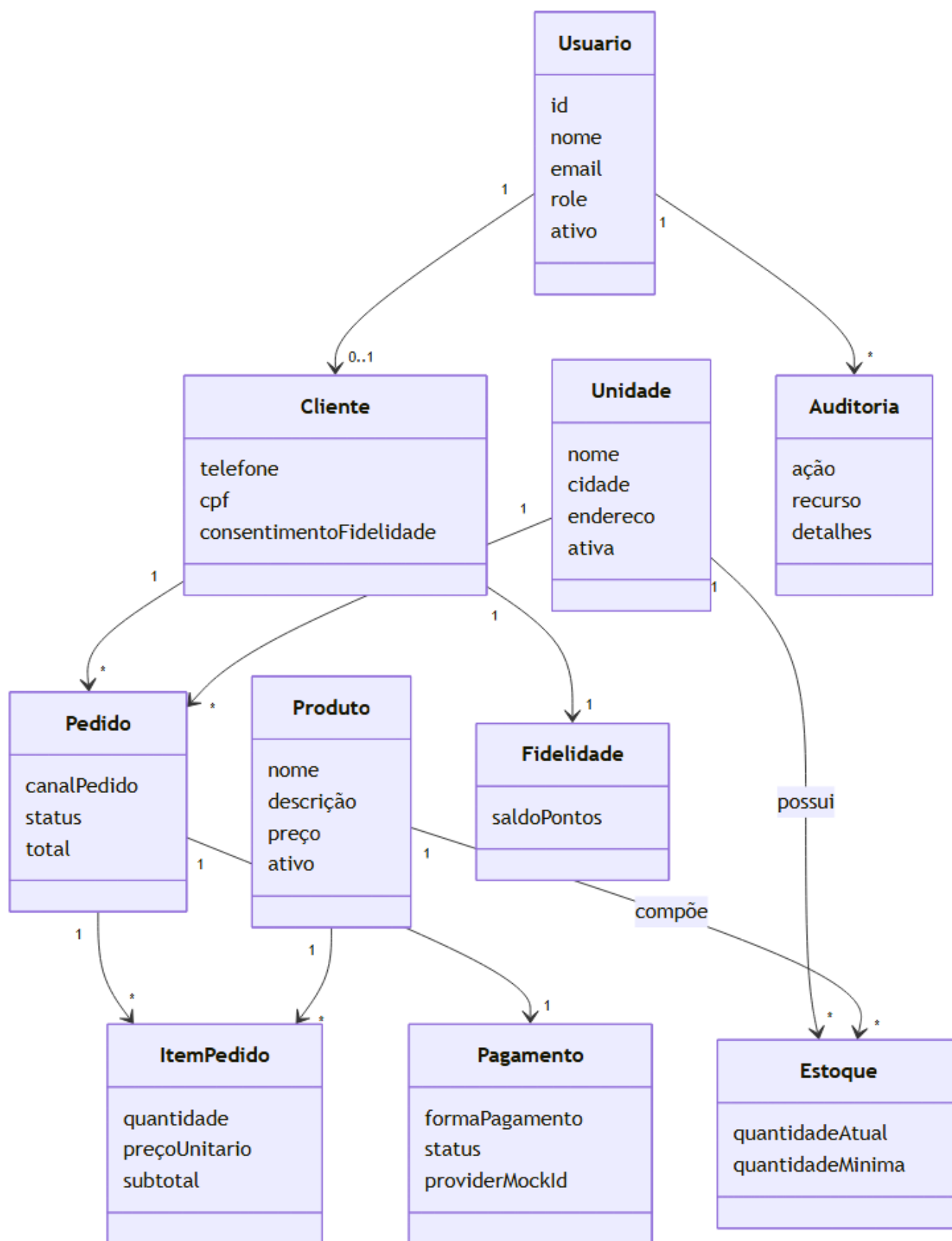
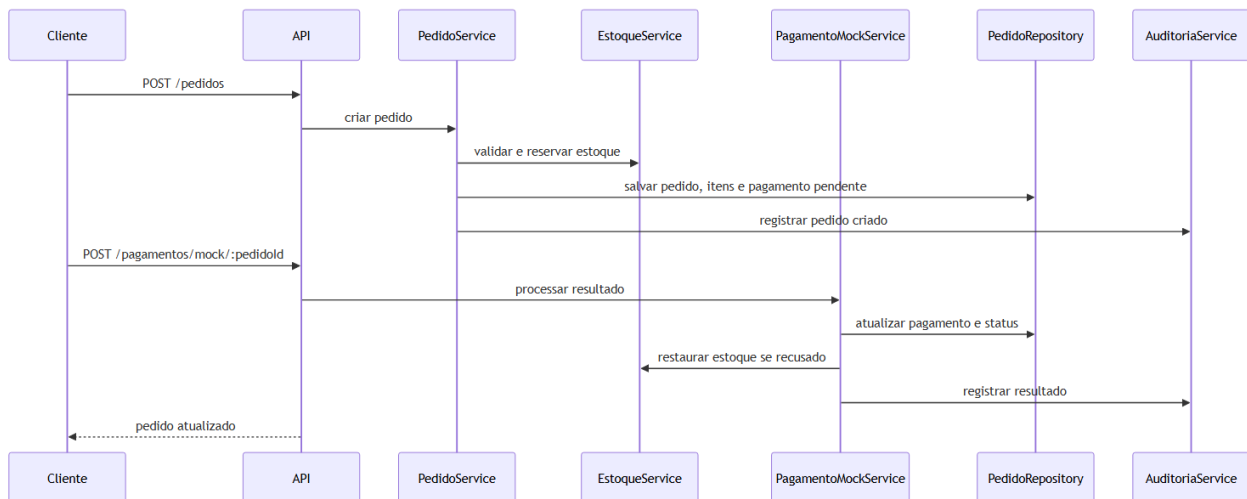


Diagrama de Sequência



Arquivo: docs/diagramas/renderizados/sequencia-pedido-pagamento.png